「天怜我，教养疏慵野性」

**Wi.** Zhimu Yang

Hydrangea Academy of Magic, HAM

Magnolia Institute of Science, MIS.HAM

# 复杂网络(1)

---

> *Type:*
>
> *~~Tutorial~~ / Note*
>
> *For:*
>
> *Amateurs / ~~Professionals~~*
>
> *Preface:*
>
> *为何又开了这一系列呢？大抵是我在学习过程中又发现，要想对神经网络有更深层次的解释，那么对于“网络”这一架构的认识不可或缺。在我的理解中，网络其实是图论的一种延伸，是将节点和边推广到极大数目而产生的统计性质，以统计物理的思想解读图论，以复杂系统的方式研究图论，所以我们这里使用名称“复杂网络”，使它同互联网的“网络”分别开来，并体现其复杂性。*
>
> *Reference:*
>
> *[1] M. E. J. Newman . 《Networks: An Introduction》. Oxford University Press, 2010*
>
> *[2]何大韧，刘宗华，汪秉宏.《复杂系统与复杂网络》. 高等教育出版社，2009*

# 一、Fundamentals

## 1.Basic

复杂网络的基础一般都是关于图论的一些知识，这里我们就只做简单介绍啦。

```
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt

#创建空无向图
G = nx.Graph()

#设置布局
pos = nx.spring_layout(G)

#绘制图
nx.draw(G, pos, with_labels=True, arrows=True)
plt.show()
```

Edge：边

```
#在顶点间添加边
G.add_egde('A', 'B')
#从容器中添加边
G.add_edge_from(cont)
```

Vertice：顶点

```
#添加节点（节点可以是任意对象，包括图）
G.add_node(obj)
#从节点集中添加节点
G.add_node_from(obj)
```

Multiedge：多重边/平行边，两个顶点直接存在多个直接连接的边

```
#创建多图
MG = nx.MultiGraph()
```

Self-Edge/Self-Loop：自环，起点和终点为同一个顶点的边

```
#创建自环
G.add_edge('A', 'A')
```

Simple Graph：简单图，不含自环和多重边的图

## Adjacency Matrix：邻接矩阵

> *Defination*
>
> *邻接矩阵是一个矩阵，它的第$i$行第$j$列元素的值代表着顶点$i$和$j$之间存在$n$条边。*
>
> $$A_{ij}=\begin{cases} n,\ i\neq j\text{且顶点}i\text{与}j\text{间存在}n\text{条边}(n=0,1,2\cdots) \\ 2n,\ i=j\text{且顶点}i\text{与其自身间存在}n\text{个自环}(n=0,1,2\cdots) \end{cases}$$

- **For a undirected graph with no self-edge,the main diagonal matrix elements are zero.**

自环对应的邻接矩阵元素为$A_{ii}$，当不存在自环，即$A_{ii}=0$，自然主对角线上的元素为0。

- **The adjacency matrix of undircted graph is symmetric.**

无向图邻接矩阵的元素$A_{ij}=n$代表顶点$i$到顶点$j$之间存在$n$条边，其实相对的，从顶点$j$到顶点$i$之间也就是$n$条边，即它的对称元素也为$A_{ji}=n$。

- **A single self-edge should be counted twice in undircted graph,which means $A_{ii}=2$ .**

从上一个性质中我们知道，无向图中从$i$到$j$与从$j$到$i$的边数相同，记$A_{ij}=n, A_{ji}=n$，无向的性质决定了这是同一条边，却在邻接矩阵中被统计两次，有重复，所以我们也应该对自环统计两次，即$A_{ii}=2$。

```
#计算邻接矩阵（返回的邻接矩阵为稀疏格式）
adj_matrix = nx.adjacency_matrix(G)
print(adj_matrix.todense())
```

## Weighted Graph：有权图

> *Defination*
>
> *我们将为图中的边赋予一个实数（可正可负）的值作为它的权重，构建有权图。显而易见的是，在我们先前的介绍中，是把一条边的权重视为1的。*

- **Weighted Graph & Multiedge Graph With the Same Adjacency Matrix.**

想象一下，两个顶点$i$和$j$之间的邻接矩阵元素为$n$（$n$大于1），有什么可能？可能是两个顶点之间有$n$条边；也可能是两个顶点之间只有1条边，但权重为$n$。这意味着正整数有权图和多重图将共享一个邻接矩阵，两种图在一定情况下可以相互转化。

```
#添加有权边
G.add_edge('A', 'B', weight=2.0)

#建立有权图
#边的属性weight、color等需要通过字典{}来指定
edges = [('A', 'B', {'weight': 2.0}),
         ('B', 'C', {'weight': 1.5}),
         ('A', 'C', {'weight': 3.0})]
G.add_edges_from(edges)
```

## Directed Graph：有向图

> *Defination*
>
> *有向图即是赋予边方向，这时从顶点$i$到$j$与顶点$j$到$i$将不再是同一条边。我们习惯上把$A_{ij}$记为从顶点$j$指向顶点$j$的元素（这与我们直觉相反）。*

- **The adjacency matrix of dircted graph is asymmetric in general.**

因为$i$到$j$与$j$到$i$不再是同一条边，所以它们对应的邻接矩阵元素也一定相同了，邻接矩阵自然也不一定对称了。

- **A single self-edge should be counted twice in undircted graph,which means $A_{ii} = 1$.**

在有向图中，由于$i$到$j$与$j$到$i$不再是同一条边，在邻接矩阵中被分开表示，对于$i$到$j$只统计一次，对于$j$到$i$也只统计一次，并没有重复，所以我们对自环也不再重复统计，即只记$A_{ii} = 1$。当然大家可能会好奇，自环逆时针和顺时针那不是也有两种方向吗？为什么不记为2？这是误解。我们对方向的定义是这样的——从一个顶点到另一个顶点为一个方向。那

么在自环中，从自己到自己，无论顺时针逆时针都只是同一个方向。

```
#创建空有向图
DG = nx.DiGraph()

#建立有向图
edges = [('A', 'B', {'weight':2.0}),
         ('B', 'C', {'weight':1.5}),
         ('A', 'C', {'weight':3.0})]
DG.add_edges_from(edges)
```

其实不用说也知道，无向图比有向图更简单，如果能把有向图转化成无向图，岂不是事半功倍？接下来我们就讨论将有向图转化为无向图的方法，直接把方向去除会使我们损失图的信息，这当然不行。

```
#直接将有向图转化成无向图
G = DG.to_undirected()
print(type(G))
```

## 2.Advanced

### Cocitation：共引

> *Defination*
>
> *顶点$i$和$j$被同样的$n$个顶点同时指向，$n$为共引数。*
>
> *或者更具体的，在引文网络中，共引指两篇文献**被**同一篇或多篇文献**同时引用**。*

- **If $i, j$ are both cited by $k$，we get $A_{ik}A_{jk} = 1$, otherwise 0.**

$i$被$k$指向，即$A_{ik} = 1$，$j$被$k$指向，即$A_{jk} = 1$，所以$A_{ik}A_{jk} = 1 \cdot 1 = 1$，只要$i, j$中有一个没有被$k$指向，则就会出现$1 \cdot 0 = 0$。

同时观察到，共引的两个顶点在邻接矩阵中是位于同一列上的，因为它们的列$k$相同。

- **The cocitation $C_{ij}$ of $i, j$ is：**

$$C_{ij} = \sum_{k=1}^{n} A_{ik}A_{jk} = \sum_{k=1}^{n} A_{ik}A_{kj}^{T}$$

共引数在邻接矩阵中表现为第$i$行与第$j$行对应元素相乘的和，这是行×行，在矩阵乘法中，我们通常将行×列。我们知道，矩阵乘法只有在第一个矩阵的列数等于第二个矩阵的行数时才能进行，所以我们将它化为矩阵乘法的形式$A_{ik}A_{kj}^T$，这么做就有助于我们推广出接下来的结论。

- **Cocitation matrix $C$ is given by：**

$$C = AA^T$$

  **where $C$ is a symmetric matrix.**

有了上次的矩阵乘法形式，把共引推广为共引矩阵也就不难理解了。共引矩阵是邻接矩阵与其转置的乘积，而且它是对称的，因为有证明：
$C^T = (AA^T)^T = (A^T)^T(A)^T = AA^T = C$。

- **The cocitation network is conventionally defined to have no self-edges, better, we can make the cocitation network a weighted network with weights $C_{ij}$ on the edges .**

共引共引，肯定是不存在自己引用自己的情况的。

- **The cocitation network is undirected.**

引文网络的共引矩阵是对称的，而无向图的邻接矩阵都是对称的，自然引文网络就是一种无向图。

- **The cocitation matrix is differ from adjacency matrix in its diagonal elements which are given by:**

$$C_{ii} = \sum_{k=1}^{n} A_{ik}^2 = \sum_{k=1}^{n} A_{ik}$$

$C_{ii}$等于所有指向$i$的边，事实上抛开主对角线元素后，邻接矩阵就与主对角线全为0的共引矩阵相同了。

## Bibliographic Coupling：文献耦合

> *Defination*
>
> *顶点$i$和$j$同时指向同样的$n$个顶点，$n$为耦合数。*
>
> *或者更具体的，在引文网络中，文献耦合指两篇文献**同时引用**同一篇或多篇文献。*

- **If $i, j$ both cite $k$，we get $A_{ki}A_{kj} = 1$, otherwise 0.**

文献耦合是共引的反向，我们就不多赘述了。

- **The bibliographic coupling of $i, j$ is:**

$$B_{ij} = \sum_{k=1}^{n} A_{ki}A_{kj} = \sum_{k=1}^{n} A_{ik}^{T}A_{kj}$$

- **Bibliographic Coupling Matrix $B$ is given by：**

$$B = A^{T}A$$

  **where $B$ is also a symmetric matrix.**

- **The diagonal elements of B is:**

$$B_{ii} = \sum_{k=1}^{n} A_{ki}^{2} = \sum_{k=1}^{n} A_{ki}$$

文献耦合与共引是两种衡量**顶点相似性**的指标。耦合可以在文献产生时就计算，共引只能在被引后才能计算。并且耦合是固定的，在文章发出后它的引用就是确定的，而共引是会随着被引而变化的。

由共引与耦合，我们能够得到全新的图搜索算法——HITS。